

数值计算方法 第三次平时作业

自动化（控制）

1 问题描述

位于两物体之间的非绝热杆，系统处于稳定状态，需要确定杆上 $[0, L]$ 的温度分布。温度满足微分方程：

$$\frac{d^2 T}{dx^2} + h'(T_a - T) = 0$$

其中 $T_a = 20$, $L = 10$ m, $h' = 0.01$ m⁻², 边界条件为 $T(0) = T_b = 40$, $T(L) = T_c = 200$ 。解析解为：

$$T = 73.4523 e^{0.1x} - 53.4523 e^{-0.1x} + 20$$

要求：取 $\Delta x = 2, 1, 0.5, 0.05$ m 时，分别采用直接法和迭代法的不同方法求解方程组并进行对比。

2 问题分析

2.1 有限差分离散化

用中心差商代替二阶导数，可得到每个内部节点的差分方程：

$$\frac{T_{i+1} - 2T_i + T_{i-1}}{\Delta x^2} + h'(T_a - T_i) = 0$$

整理得：

$$-T_{i-1} + (2 + h'\Delta x^2) T_i - T_{i+1} = h'\Delta x^2 T_a$$

记 $d = 2 + h'\Delta x^2$, $r = h'\Delta x^2 T_a$, 将区间 $[0, L]$ 等分为 n 段 ($n = L/\Delta x$)，则有 $m = n - 1$ 个内部未知点。边界值 $T_0 = T_b$ 、 $T_n = T_c$ 为已知，将其移至右端向量，得到三对角线性方程组 $AT = b$ ：

$$\begin{bmatrix} d & -1 & & \\ -1 & d & -1 & \\ & \ddots & \ddots & \ddots \\ & & -1 & d \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ \vdots \\ T_m \end{bmatrix} = \begin{bmatrix} r + T_b \\ r \\ \vdots \\ r + T_c \end{bmatrix}$$

以 $\Delta x = 2$ 为例， $d = 2 + 0.01 \times 4 = 2.04$, $r = 0.01 \times 4 \times 20 = 0.8$, 方程组为 4×4 的三对角系统。

2.2 系数矩阵的性质

该系数矩阵具有以下重要性质：(1) 三对角矩阵，适合用追赶法高效求解；(2) 严格对角占优 ($|d| = 2 + h'\Delta x^2 > 2 \geq |-1| + |-1|$)，因此 Jacobi 和 Gauss-Seidel 迭代法均保证收敛；(3) 对称正定矩阵，可使用 Cholesky 分解，同时保证 SOR 在 $0 < \omega < 2$ 时收敛。

2.3 求解方法

采用 6 种方法求解：直接法包括追赶法 (Thomas 算法)、高斯列主元消去法和 LU 分解 (Doolittle)；迭代法包括 Jacobi 迭代、Gauss-Seidel 迭代和 SOR 迭代。迭代法的终止条件为相对误差准则 $\|x^{(k+1)} - x^{(k)}\|_\infty / \|x^{(k+1)}\|_\infty < 10^{-6}$ ，最大迭代次数设为 100000。

3 追赶法 (Thomas 算法)

3.1 算法分析

追赶法是针对三对角方程组的高效直接法，本质上是 Doolittle LU 分解在三对角结构下的简化。算法分为两步：“追”（前向消去）从第 2 行开始逐行消去下次对角线元素，更新主对角线和右端项；“赶”（回代）从最后一行开始逐行回代求解。由于系数矩阵严格对角占优，消去过程中主对角线元素始终为最大，因此不需要选主元，数值稳定。计算量仅为 $5m - 4$ 次运算，远小于一般高斯消去法的 $O(m^3/3)$ 。

3.2 MATLAB 程序

程序中首先构建三对角方程组，然后调用追赶法求解。完整代码附在报告最后，此处仅展示追赶法核心代码。

```
function x = thomas(A, b)
    m = length(b); d = diag(A); f = b;
    % 追（前向消去）
    for i = 2:m
        w = A(i, i-1) / d(i-1);
        d(i) = d(i) - w * A(i-1, i);
        f(i) = f(i) - w * f(i-1);
    end
    % 赶（回代）
    x = zeros(m, 1); x(m) = f(m) / d(m);
    for i = m-1:-1:1
        x(i) = (f(i) - A(i, i+1) * x(i+1)) / d(i);
    end
end
```

3.3 运行结果及分析

```
dx=2.00: 最大相对误差 = 3.269950e-04
dx=1.00: 最大相对误差 = 8.191490e-05
dx=0.50: 最大相对误差 = 2.035790e-05
dx=0.05: 最大相对误差 = 2.473117e-07
```

追赶法作为直接法，在有限步内即可得到精确解（仅受浮点舍入误差影响）。随着 Δx 减小，离散化误差降低，最大相对误差从 3.27×10^{-4} 降至 2.47×10^{-7} ，近似呈 $O(\Delta x^2)$ 的收敛趋势，与中心差分格式的二阶精度一致。

4 高斯列主元消去法

4.1 算法分析

高斯列主元消去法是通用的直接法，适用于任意非奇异方程组。算法在每步前向消去前，先在当前列的剩余行中选取绝对值最大的元素作为主元并交换行，以避免小主元导致的数值不稳定。消去完成后得到上三角矩阵，再通过回代求解。对于本题的三对角矩阵，高斯消去法的大量操作花在处理零元素上，效率不如追赶法，但其通用性更强。

4.2 MATLAB 程序

完整代码附在报告最后，此处仅展示高斯列主元消去法核心代码。

```
function x = gauss_pivot(A, b)
    m = length(b); Aug = [A, b];
    for k = 1:m-1
        [~, idx] = max(abs(Aug(k:m, k)));
        p = idx + k - 1;
        if p ~= k, Aug([k,p],:) = Aug([p,k],:); end
        for i = k+1:m
            factor = Aug(i,k) / Aug(k,k);
            Aug(i, k:end) = Aug(i, k:end) - factor * Aug(k, k:end);
        end
    end
    x = zeros(m, 1); x(m) = Aug(m, m+1) / Aug(m, m);
    for i = m-1:-1:1
        x(i) = (Aug(i,m+1) - Aug(i,i+1:m)*x(i+1:m)) / Aug(i,i);
    end
end
```

4.3 运行结果及分析

```

dx=2.00: 最大相对误差 = 3.269950e-04
dx=1.00: 最大相对误差 = 8.191490e-05
dx=0.50: 最大相对误差 = 2.035790e-05
dx=0.05: 最大相对误差 = 2.473117e-07

```

高斯列主元消去法的结果与追赶法完全一致，验证了两种直接法在数值精度上的等价性。对于本题的对角占优矩阵，列主元选取实际上不会触发行交换（主对角线元素已经是当前列最大的），但列主元机制为一般方程组提供了额外的数值稳定性保障。

5 LU 分解 (Doolittle)

5.1 算法分析

Doolittle LU 分解将系数矩阵分解为 $A = LU$ ，其中 L 为单位下三角矩阵， U 为上三角矩阵。求解过程分为三步：分解 $A = LU$ 、前代求解 $Ly = b$ 、回代求解 $Ux = y$ 。LU 分解的优势在于分解只需执行一次，对不同的右端向量 b 只需重新执行代入步骤 ($O(m^2)$)，这在需要求解同一系数矩阵、多个右端向量的问题中非常高效。

5.2 MATLAB 程序

完整代码附在报告最后，此处仅展示 LU 分解核心代码。

```

function x = lu_solve(A, b)
    m = length(b); L_mat = eye(m); U_mat = A;
    for k = 1:m-1
        for i = k+1:m
            L_mat(i,k) = U_mat(i,k) / U_mat(k,k);
            U_mat(i,k:end) = U_mat(i,k:end) - L_mat(i,k)*U_mat(k,k:end);
        end
    end
    y = zeros(m,1); y(1) = b(1);
    for i = 2:m
        y(i) = b(i) - L_mat(i,1:i-1) * y(1:i-1);
    end
    x = zeros(m,1); x(m) = y(m) / U_mat(m,m);
    for i = m-1:-1:1
        x(i) = (y(i) - U_mat(i,i+1:m)*x(i+1:m)) / U_mat(i,i);
    end
end

```

5.3 运行结果及分析

```

dx=2.00: 最大相对误差 = 3.269950e-04
dx=1.00: 最大相对误差 = 8.191490e-05
dx=0.50: 最大相对误差 = 2.035790e-05
dx=0.05: 最大相对误差 = 2.473117e-07

```

LU 分解的结果同样与追赶法和高斯消去法完全一致。这三种直接法的数值误差完全来自有限差分的离散化，而非求解过程本身。实际上，追赶法正是 Doolittle LU 分解针对三对角矩阵的简化版本，两者在数学上等价。

6 Jacobi 迭代法

6.1 算法分析

Jacobi 迭代法是一种同步迭代法：在计算第 $k+1$ 步的所有分量时，全部使用第 k 步的旧值。对于本题的方程组，迭代公式为：

$$T_i^{(k+1)} = \frac{1}{A(i,i)} \left(b(i) - \sum_{j \neq i} A(i,j) \cdot T_j^{(k)} \right)$$

由于系数矩阵严格对角占优，Jacobi 迭代保证收敛。但由于每步迭代不使用已经更新的分量，收敛速度相对较慢，且需要两组存储空间（旧值和新值）。

6.2 MATLAB 程序

完整代码附在报告最后，此处仅展示 Jacobi 迭代核心代码。

```

function [x, iter] = jacobi(A, b, tol, max_iter)
    m = length(b); x_old = zeros(m, 1);
    for iter = 1:max_iter
        x_new = zeros(m, 1);
        for i = 1:m
            sigma = 0;
            for j = 1:m
                if j ~= i, sigma = sigma + A(i,j)*x_old(j); end
            end
            x_new(i) = (b(i) - sigma) / A(i,i);
        end
        if norm(x_new-x_old,inf)/norm(x_new,inf) < tol
            x = x_new; return;
        end
        x_old = x_new;
    end
    x = x_new;
end

```

6.3 运行结果及分析

```
dx=2.00: 最大相对误差 = 3.235623e-04, 迭代次数 = 56
dx=1.00: 最大相对误差 = 6.591260e-05, 迭代次数 = 206
dx=0.50: 最大相对误差 = 5.153550e-05, 迭代次数 = 727
dx=0.05: 最大相对误差 = 7.596631e-03, 迭代次数 = 38661
```

Jacobi 迭代法在 Δx 较大时能较快收敛，但随着 Δx 减小、矩阵规模增大，迭代次数急剧增长。当 $\Delta x = 0.05$ 时，Jacobi 法在第 38661 次迭代时已满足迭代终止准则 $\|x^{(k+1)} - x^{(k)}\|_\infty / \|x^{(k+1)}\|_\infty < 10^{-6}$ ，但此时相对于解析解的最大相对误差仍有 7.6×10^{-3} 。这说明当矩阵规模较大时，仅以相邻两次迭代差作为停止标准，可能会使迭代在“变化已很小”时停止，而与解析解之间仍存在较明显偏差。其根本原因在于 Jacobi 迭代矩阵的谱半径 ρ_J 随矩阵规模增大而趋近于 1，导致收敛速度很慢。

7 Gauss-Seidel 迭代法

7.1 算法分析

Gauss-Seidel (G-S) 迭代法是一种异步迭代法：在计算 $T_i^{(k+1)}$ 时，立即使用已经计算出的 $T_1^{(k+1)}, \dots, T_{i-1}^{(k+1)}$ ，而非全部使用旧值。迭代公式为：

$$T_i^{(k+1)} = \frac{1}{A(i,i)} \left(b(i) - \sum_{j<i} A(i,j) T_j^{(k+1)} - \sum_{j>i} A(i,j) T_j^{(k)} \right)$$

相比 Jacobi 方法，G-S 方法只需一组存储空间，且通常收敛更快。对于严格对角占优矩阵，G-S 方法同样保证收敛。

7.2 MATLAB 程序

完整代码附在报告最后，此处仅展示 G-S 迭代核心代码。

```
function [x, iter] = gauss_seidel(A, b, tol, max_iter)
    m = length(b); x = zeros(m, 1);
    for iter = 1:max_iter
        x_old = x;
        for i = 1:m
            sigma = 0;
            for j = 1:m
                if j ~= i, sigma = sigma + A(i,j)*x(j); end
            end
            x(i) = (b(i) - sigma) / A(i,i);
        end
        if norm(x-x_old,inf)/norm(x,inf) < tol, return; end
    end
end
```

```
end
```

7.3 运行结果及分析

```
dx=2.00: 最大相对误差 = 3.249500e-04, 迭代次数 = 30
dx=1.00: 最大相对误差 = 6.690687e-05, 迭代次数 = 106
dx=0.50: 最大相对误差 = 5.271176e-05, 迭代次数 = 368
dx=0.05: 最大相对误差 = 7.563389e-03, 迭代次数 = 19395
```

G-S 方法的迭代次数约为 Jacobi 方法的一半，体现了即时使用更新分量带来的加速效果。但在 $\Delta x = 0.05$ 时，G-S 法虽然在第 19395 次迭代时已满足迭代终止准则，但相对于解析解的最大相对误差仍为 7.563×10^{-3} ，说明对于大规模问题，仅靠 G-S 的改进仍不足以从根本上解决收敛速度偏慢的问题。

8 SOR 迭代法

8.1 算法分析

逐次超松弛（SOR）方法在 G-S 迭代基础上引入松弛因子 ω ：

$$T_i^{(k+1)} = (1 - \omega) T_i^{(k)} + \omega \tilde{T}_i^{(k+1)}$$

其中 $\tilde{T}_i^{(k+1)}$ 为 G-S 迭代算出的值。当 $\omega = 1$ 时退化为 G-S 方法； $1 < \omega < 2$ 为超松弛，可加速收敛。

最优松弛因子可由 Jacobi 迭代矩阵的谱半径 ρ_J 确定：

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \rho_J^2}}$$

对于不同的 Δx ， ρ_J 和 ω_{opt} 的值如下： $\Delta x = 2$ 时 $\omega_{\text{opt}} = 1.243$ ； $\Delta x = 1$ 时 $\omega_{\text{opt}} = 1.512$ ； $\Delta x = 0.5$ 时 $\omega_{\text{opt}} = 1.718$ ； $\Delta x = 0.05$ 时 $\omega_{\text{opt}} = 1.968$ 。可以看到， Δx 越小， ω_{opt} 越接近 2，SOR 的加速效果也越显著。

8.2 MATLAB 程序

完整代码附在报告最后，此处仅展示 SOR 迭代核心代码。

```
function [x, iter] = sor(A, b, omega, tol, max_iter)
    m = length(b); x = zeros(m, 1);
    for iter = 1:max_iter
        x_old = x;
        for i = 1:m
            sigma = 0;
            for j = 1:m
```

```

        if j ~= i, sigma = sigma + A(i,j)*x(j); end
    end
    gs_val = (b(i) - sigma) / A(i,i);
    x(i) = (1 - omega)*x_old(i) + omega*gs_val;
end
if norm(x-x_old,inf)/norm(x,inf) < tol, return; end
end
end
end

```

8.3 运行结果及分析

$\Delta x=2.00$: 最大相对误差 = $3.267716e-04$, $w=1.2430$, 迭代次数 = 14
 $\Delta x=1.00$: 最大相对误差 = $8.044685e-05$, $w=1.5115$, 迭代次数 = 27
 $\Delta x=0.50$: 最大相对误差 = $1.766070e-05$, $w=1.7182$, 迭代次数 = 53
 $\Delta x=0.05$: 最大相对误差 = $8.382061e-05$, $w=1.9676$, 迭代次数 = 442

SOR 方法在所有 Δx 下均表现优异。特别是在 $\Delta x = 0.05$ 时, SOR 仅需 442 次迭代即可收敛, 而 Jacobi 和 G-S 虽然分别在 38661 次和 19395 次迭代时满足了停止准则, 但相对于解析解的误差仍明显偏大, SOR 的加速效果极为显著。

以 $\Delta x = 2$ 为例, 不同 ω 值对迭代次数的影响如下表所示:

ω	0.8	1.0	1.1	1.2	1.3	1.5	1.7	1.9
迭代次数	45	30	24	17	14	23	41	135

表 1: $\Delta x = 2$ 时不同松弛因子 ω 的迭代次数

可以看到 $\omega \approx 1.3$ 时迭代次数最少 (14 次), 与理论最优值 $\omega_{\text{opt}} = 1.243$ 接近。 ω 偏离最优值时迭代次数增加, ω 接近 2 时收敛急剧变慢。图 1 更直观地展示了这一关系。

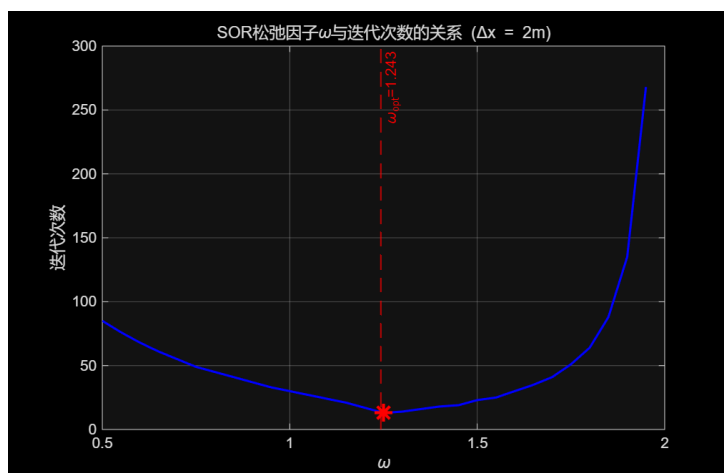


图 1: SOR 松弛因子 ω 与迭代次数的关系 ($\Delta x = 2m$)

9 比较总结

9.1 汇总对比表

方法	$\Delta x = 2$	$\Delta x = 1$	$\Delta x = 0.5$	$\Delta x = 0.05$
最大相对误差				
追赶法	3.27×10^{-4}	8.19×10^{-5}	2.04×10^{-5}	2.47×10^{-7}
高斯列主元	3.27×10^{-4}	8.19×10^{-5}	2.04×10^{-5}	2.47×10^{-7}
LU 分解	3.27×10^{-4}	8.19×10^{-5}	2.04×10^{-5}	2.47×10^{-7}
Jacobi	3.24×10^{-4}	6.59×10^{-5}	5.15×10^{-5}	7.60×10^{-3}
G-S	3.25×10^{-4}	6.69×10^{-5}	5.27×10^{-5}	7.56×10^{-3}
SOR	3.27×10^{-4}	8.04×10^{-5}	1.77×10^{-5}	8.38×10^{-5}
迭代次数（直接法不适用）				
Jacobi	56	206	727	38661
G-S	30	106	368	19395
SOR	14	27	53	442

表 2: 六种方法在不同步长下的对比结果

9.2 精度分析

三种直接法的最大相对误差完全一致，且随步长减小呈 $O(\Delta x^2)$ 的下降趋势，这与中心差分格式的二阶精度一致。迭代法在充分收敛后能达到与直接法相当的精度，但当矩阵规模较大、停止准则仅基于相邻迭代差时（如 $\Delta x = 0.05$ 的 Jacobi 和 G-S），即使程序已经停止，结果相对于解析解的误差仍可能显著偏大。

图 2 展示了直接法的最大相对误差随 Δx 减小的变化趋势，在双对数坐标下近似为斜率为 2 的直线，验证了二阶收敛精度。

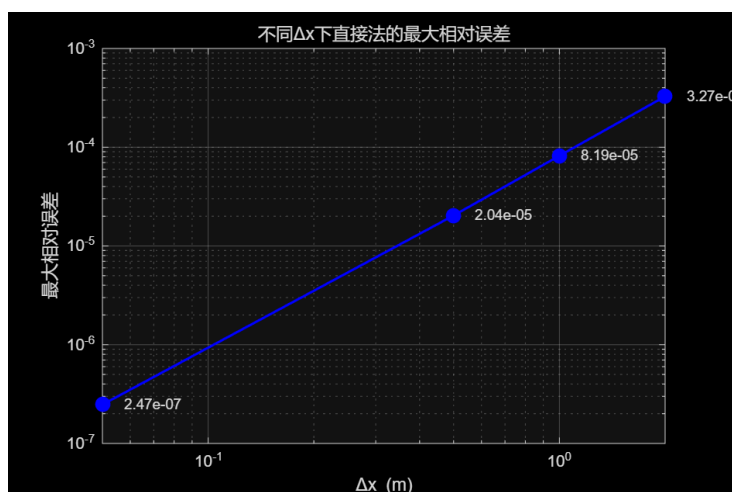


图 2: 不同 Δx 下直接法的最大相对误差

9.3 收敛速度

三种迭代法中，SOR 的收敛速度最快。G-S 方法的迭代次数约为 Jacobi 的一半，这与理论分析一致。随着 Δx 减小，Jacobi 和 G-S 的迭代次数急剧增长（与 $1/\Delta x^2$ 近似成正比），而 SOR 通过选取最优松弛因子有效抑制了这一增长。图 3 直观地展示了这一差异。

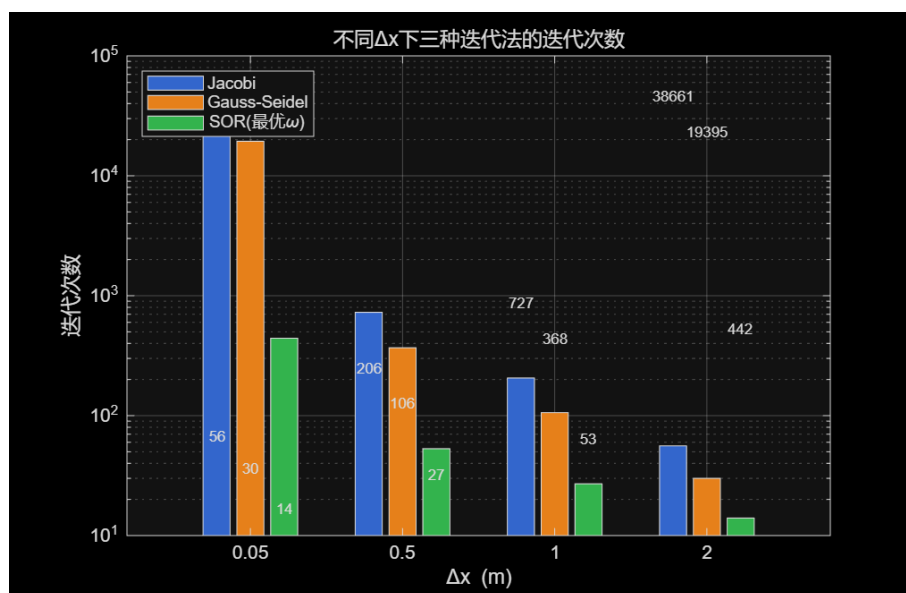


图 3: 不同 Δx 下三种迭代法的迭代次数对比（对数坐标）

9.4 计算效率

对于本题的三对角矩阵，追赶法是最高效的选择，计算量仅为 $O(m)$ 。高斯消去法和 LU 分解的计算量为 $O(m^3/3)$ ，对于大规模问题效率远低于追赶法。迭代法的单步计算量为 $O(m^2)$ （通用实现）或 $O(m)$ （利用三对角结构），但总计算量取决于迭代次数，在大规模问题中可能远超直接法。

9.5 适用性

追赶法仅适用于三对角方程组；高斯消去法和 LU 分解适用于一般稠密矩阵；迭代法适用于大型稀疏矩阵（需满足收敛条件）。对于本题这类三对角问题，追赶法是首选；当矩阵结构未知时，高斯列主元消去法是最稳健的通用方法；对于超大规模稀疏问题，SOR 迭代法（配合最优松弛因子）是实用的选择。

10 附：完整 MATLAB 程序及运行结果

10.1 完整 MATLAB 程序

```
% 数值计算方法 第三章作业 - 非绝热杆温度分布求解
clear; clc;
```

```

%% 物理参数
Ta = 20; Tb = 40; Tc = 200; L = 10; h_coeff = 0.01;
T_exact = @(x) 73.4523*exp(0.1*x) - 53.4523*exp(-0.1*x) + 20;
tol = 1e-6; max_iter = 100000;
dx_list = [2, 1, 0.5, 0.05];

%% 主循环
for idx = 1:length(dx_list)
    dx = dx_list(idx);
    n = L / dx; m = n - 1;
    fprintf('dx = %.2f m, m = %d\n', dx, m);

    [A, b] = build_system(dx, Ta, Tb, Tc, L, h_coeff);
    x_nodes = (dx:dx:L-dx)';
    T_ana = T_exact(x_nodes);

    T1 = thomas(A, b);
    err1 = max(abs((T1 - T_ana) ./ T_ana));
    fprintf('追赶法: 误差 = %.6e\n', err1);

    T2 = gauss_pivot(A, b);
    err2 = max(abs((T2 - T_ana) ./ T_ana));
    fprintf('高斯列主元: 误差 = %.6e\n', err2);

    T3 = lu_solve(A, b);
    err3 = max(abs((T3 - T_ana) ./ T_ana));
    fprintf('LU分解: 误差 = %.6e\n', err3);

    [T4, iter4] = jacobi(A, b, tol, max_iter);
    err4 = max(abs((T4 - T_ana) ./ T_ana));
    fprintf('Jacobi: 误差 = %.6e, 迭代 = %d\n', err4, iter4);

    [T5, iter5] = gauss_seidel(A, b, tol, max_iter);
    err5 = max(abs((T5 - T_ana) ./ T_ana));
    fprintf('G-S: 误差 = %.6e, 迭代 = %d\n', err5, iter5);

    J = diag(diag(A)) \ (diag(diag(A)) - A);
    rho_J = max(abs(eig(J)));
    omega_opt = 2 / (1 + sqrt(1 - rho_J^2));
    [T6, iter6] = sor(A, b, omega_opt, tol, max_iter);
    err6 = max(abs((T6 - T_ana) ./ T_ana));
    fprintf('SOR(w=%.4f): 误差 = %.6e, 迭代 = %d\n', ...
            omega_opt, err6, iter6);
end

```

```

%% 构建方程组
function [A, b] = build_system(dx, Ta, Tb, Tc, L, h)
    n = L/dx; m = n-1;
    d = 2 + h*dx^2; r = h*dx^2*Ta;
    main = d*ones(1,m); off = -1*ones(1,m-1);
    A = diag(main,0) + diag(off,1) + diag(off,-1);
    b = r*ones(m,1); b(1) = b(1)+Tb; b(end) = b(end)+Tc;
end

%% 追赶法
function x = thomas(A, b)
    m = length(b); d = diag(A); f = b;
    for i = 2:m
        w = A(i,i-1)/d(i-1);
        d(i) = d(i) - w*A(i-1,i);
        f(i) = f(i) - w*f(i-1);
    end
    x = zeros(m,1); x(m) = f(m)/d(m);
    for i = m-1:-1:1
        x(i) = (f(i) - A(i,i+1)*x(i+1)) / d(i);
    end
end

%% 高斯列主元消去法
function x = gauss_pivot(A, b)
    m = length(b); Aug = [A, b];
    for k = 1:m-1
        [~, idx] = max(abs(Aug(k:m, k)));
        p = idx + k - 1;
        if p ~= k, Aug([k,p],:) = Aug([p,k],:); end
        for i = k+1:m
            factor = Aug(i,k)/Aug(k,k);
            Aug(i,k:end) = Aug(i,k:end) - factor*Aug(k,k:end);
        end
    end
    x = zeros(m,1); x(m) = Aug(m,m+1)/Aug(m,m);
    for i = m-1:-1:1
        x(i) = (Aug(i,m+1) - Aug(i,i+1:m)*x(i+1:m)) / Aug(i,i);
    end
end

%% LU分解
function x = lu_solve(A, b)
    m = length(b); L_mat = eye(m); U_mat = A;

```

```

    for k = 1:m-1
        for i = k+1:m
            L_mat(i,k) = U_mat(i,k)/U_mat(k,k);
            U_mat(i,k:end) = U_mat(i,k:end) - L_mat(i,k)*U_mat(k,k:end);
        end
    end
    y = zeros(m,1); y(1) = b(1);
    for i = 2:m
        y(i) = b(i) - L_mat(i,1:i-1)*y(1:i-1);
    end
    x = zeros(m,1); x(m) = y(m)/U_mat(m,m);
    for i = m-1:-1:1
        x(i) = (y(i) - U_mat(i,i+1:m)*x(i+1:m)) / U_mat(i,i);
    end
end

%% Jacobi迭代
function [x, iter] = jacobi(A, b, tol, max_iter)
    m = length(b); x_old = zeros(m,1);
    for iter = 1:max_iter
        x_new = zeros(m,1);
        for i = 1:m
            sigma = 0;
            for j = 1:m
                if j ~= i, sigma = sigma + A(i,j)*x_old(j); end
            end
            x_new(i) = (b(i)-sigma)/A(i,i);
        end
        if norm(x_new-x_old,inf)/norm(x_new,inf) < tol
            x = x_new; return;
        end
        x_old = x_new;
    end
    x = x_new;
end

%% Gauss-Seidel迭代
function [x, iter] = gauss_seidel(A, b, tol, max_iter)
    m = length(b); x = zeros(m,1);
    for iter = 1:max_iter
        x_old = x;
        for i = 1:m
            sigma = 0;
            for j = 1:m
                if j ~= i, sigma = sigma + A(i,j)*x(j); end
            end
            x(i) = (b(i)-sigma)/A(i,i);
        end
        if norm(x-x_old,inf)/norm(x,inf) < tol
            break;
        end
    end
    x = x;
end

```

```

        end
        x(i) = (b(i)-sigma)/A(i,i);
    end
    if norm(x-x_old,inf)/norm(x,inf) < tol, return; end
end
end

%% SOR迭代
function [x, iter] = sor(A, b, omega, tol, max_iter)
    m = length(b); x = zeros(m,1);
    for iter = 1:max_iter
        x_old = x;
        for i = 1:m
            sigma = 0;
            for j = 1:m
                if j ~= i, sigma = sigma + A(i,j)*x(j); end
            end
            gs_val = (b(i)-sigma)/A(i,i);
            x(i) = (1-omega)*x_old(i) + omega*gs_val;
        end
        if norm(x-x_old,inf)/norm(x,inf) < tol, return; end
    end
end
end

```

10.2 完整运行结果

```

dx = 2.00 m, m = 4
追赶法 误差 = 3.269950e-04
高斯列主元 误差 = 3.269950e-04
LU分解: 误差 = 3.269950e-04
Jacobi: 误差 = 3.235623e-04, 迭代 = 56
G-S: 误差 = 3.249500e-04, 迭代 = 30
SOR(w=1.2430): 误差 = 3.267716e-04, 迭代 = 14
dx = 1.00 m, m = 9
追赶法 误差 = 8.191490e-05
高斯列主元 误差 = 8.191490e-05
LU分解: 误差 = 8.191490e-05
Jacobi: 误差 = 6.591260e-05, 迭代 = 206
G-S: 误差 = 6.690687e-05, 迭代 = 106
SOR(w=1.5115): 误差 = 8.044685e-05, 迭代 = 27
dx = 0.50 m, m = 19
追赶法 误差 = 2.035790e-05
高斯列主元 误差 = 2.035790e-05
LU分解: 误差 = 2.035790e-05

```

```
Jacobi: 误差 = 5.153550e-05, 迭代 = 727  
G-S: 误差 = 5.271176e-05, 迭代 = 368  
SOR(w=1.7182): 误差 = 1.766070e-05, 迭代 = 53  
dx = 0.05 m, m = 199  
追赶法 误差 = 2.473117e-07  
高斯列主元 误差 = 2.473117e-07  
LU分解: 误差 = 2.473117e-07  
Jacobi: 误差 = 7.596631e-03, 迭代 = 38661  
G-S: 误差 = 7.563389e-03, 迭代 = 19395  
SOR(w=1.9676): 误差 = 8.382061e-05, 迭代 = 442
```